

Practical Work Report

Course: Deep Learning for Computer Vision

Project: Modernizing DeepSORT — Pedestrian Tracking on Video

1. Introduction

The task of Multiple Object Tracking (MOT) is defined as follows: given a video, it is required to assign a unique identifier to each object (in this case, a person) and track its movement over time. The result is a set of trajectories, each representing a sequence of bounding boxes (bbox, bounding box) sharing a common identifier.

Practical applications of MOT include surveillance systems, sports analytics, autonomous vehicles, and crowd analysis. The primary difficulty of the task lies in the need to associate detections of the same object across frames in the presence of occlusions (mutual overlap), similar appearance of different objects, and scale variability.

DeepSORT (2017) was selected as the baseline algorithm — an extension of the SORT method that adds appearance analysis via a Re-Identification (REID) model to the motion model (Kalman filter). The REID model extracts a numerical vector — an appearance descriptor — from each detected object; descriptor similarity is used for inter-frame association.

The original DeepSORT employs outdated components: the POI (Pedestrian Occupancy Information) detector and the mars-small128 REID model implemented on TensorFlow 1.x. In the context of this project, the TensorFlow model is unavailable, rendering the appearance analysis component non-functional.

The objective of this work is to modernize DeepSORT by replacing the detector and REID model with contemporary neural network architectures, followed by parameter tuning. Additionally, an autonomous body REID system with a descriptor database and temporal voting has been implemented.

The work was carried out in six stages: 1. Baseline establishment 2. Integration and comparison of detectors 3. Integration and comparison of REID models 4. Integration into a full pipeline, parameter tuning 5. Implementation of autonomous body REID 6. Generation of overlay videos, report compilation

2. Task Description

2.1. Problem Statement

Formally: given a video of N frames, it is necessary to detect all persons in each frame and assign each a unique identifier (ID) that persists over time. The output is a file in MOT format: each line contains `frame_number`, `ID`, `x`, `y`, `width`, `height`, where (x, y) are the coordinates of the top-left corner of the bbox (bounding box).

2.2. Complicating Factors

- **Occlusions** — mutual overlap of objects leads to temporary loss of observation; the algorithm must recover IDs after reappearance.
- **Appearance similarity** — REID descriptors for different objects may be close in the feature space.
- **Scale variability** — the same object may occupy from 10 to 500 pixels depending on the distance to the camera.
- **Fast motion** — object displacement between frames may exceed the Kalman filter prediction.

2.3. Evaluation Metrics

HOTA (Higher Order Tracking Accuracy) — the primary metric. It represents the geometric mean of two components: - DetJ (Detection Jaccard — detection quality) — a similarity measure between detected and ground-truth bboxes (Jaccard coefficient, i.e., intersection over union). - AssA (Association Accuracy) — the quality of linking detections of the same object across frames.

For modern trackers, HOTA ranges from 50 to 60. In this project, the baseline is 4.73.

MOTA (Multiple Object Tracking Accuracy) — the ratio of correct detections minus false positives (FP) and false negatives (FN) to the total number of ground-truth objects. Measured in percentages; a value of 50+ is considered satisfactory.

IDF1 (Identity F1) — the harmonic mean of identification precision and recall. A low IDF1 indicates frequent ID switches for a single object.

FPS (Frames Per Second) — processing speed. The assignment requires at least 5 FPS.

2.4. Test Sequences

Six video sequences from MOT Challenge — a standard benchmark for tracking tasks — were used:

Video	Frames	Resolution	GT IDs	Characteristics
TUD-Campus	71	640×480	8	Low density, simple scenario
TUD-Stadtmitte	179	640×480	10	Street scene, moderate difficulty
KITTI-17	145	1224×370	9	Dashcam recording, wide resolution
PETS09-S2L1	795	768×576	19	Moderate density, long sequence
MOT16-09	525	1920×1080	43	High density, distant objects
MOT16-11	900	1920×1080	80	High density, most challenging

The selection of sequences covers a range from simple (TUD-Campus, 8 objects) to complex (MOT16-11, 80 objects), allowing assessment of algorithm robustness under varying conditions.

3. Baseline

3.1. Experimental Conditions

The original DeepSORT was run with the following parameters: - Detections: pre-loaded `det.txt` from MOT Challenge (POI detector) - REID features: random 128-dimensional vectors (original `mars-small128.pb` model unavailable) - Environment: Python 3.13, CPU, numpy 1.26, scipy 1.17, opencv 4.11

3.2. POI Detector

POI (Pedestrian Occupancy Information) is a pedestrian detector based on aggregated channel features (ACF). It is a classical (non-neural-network) detection method used in MOT Challenge in 2016. By modern standards, it is characterized by low recall and a high number of false detections.

3.3. REID Model Issue

The original DeepSORT REID model — `mars-small128.pb` — is a small CNN (Convolutional Neural Network) trained on the MARS dataset. The model is implemented on TensorFlow 1.x, which is incompatible with Python 3.13. Conversion attempts were unsuccessful. Random 128-dimensional vectors were used as a substitute, rendering the appearance-based association component non-functional — the algorithm relies exclusively on the motion model (Kalman filter).

3.4. Results

Video	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	7.95	41.48	1.53	33.98	36.82	280.71
TUD-Stadtmitte	5.33	44.11	0.64	38.58	45.47	306.25
KITTI-17	5.93	40.56	0.87	33.63	43.79	356.42
PETS09-S2L1	4.08	60.98	0.27	51.96	49.29	232.78
MOT16-09	2.59	28.73	0.23	24.59	17.78	346.47
MOT16-11	2.52	38.61	0.16	36.15	22.60	484.76
Average	4.73	—	—	36.48	35.96	334.57

The low HOTA value of 4.73 is attributed to two factors: AssA (association accuracy) is near zero (0.16–1.53) due to random REID features; DetJ is low (28–61) due to the outdated detector. The high FPS (334.57) is explained by the absence of neural network inference — only the Kalman filter and the Hungarian algorithm are executed.

4. Detector Integration

4.1. Methodology

YOLO (You Only Look Once) is a family of single-stage neural networks for object detection. Unlike classical sliding-window methods, YOLO processes the entire image in a single pass and predicts a bbox and class for each object. The choice is motivated by high detection quality, the availability of the Ultralytics library (PyTorch), and support for the person class from the COCO dataset.

A unified interface `detectors/base_detector.py` with a factory function `create_detector(name)` was created, enabling detector switching via a command-line parameter. All detectors were configured identically: `conf_threshold=0.3`, `img_size=640`, `class person`.

4.2. Compared Models

Model	Family	mAP (COCO)	Parameters
YOLO11n	YOLO11	39.5	2.6M
YOLO11s	YOLO11	47.0	9.4M
YOLOv8n	YOLOv8	37.3	3.2M
YOLOv8s	YOLOv8	44.9	11.2M

4.3. Results

Detector	Precision	Recall	F1	FPS
yolo11s	0.858	0.641	0.734	8.77
yolo11n	0.858	0.641	0.734	8.57
yolov8n	0.858	0.641	0.734	8.41
yolov8s	0.858	0.641	0.734	5.76

The identical precision, recall, and F1 values are explained by the fact that all models detect the same persons on the test data. Differences manifest in speed (FPS) and detection quality on complex scenes (mAP).

4.4. Selection

The **YOLO11s** model was selected: highest mAP (47.0), high FPS (8.77 on CPU), segmentation support (YOLO11-seg variant), modern architecture (2024).

5. REID Model Integration

5.1. Purpose of REID

REID (Re-Identification) is the task of recognizing an object by its appearance upon reappearance in the frame. In the context of tracking, a REID model extracts a numerical vector (appearance descriptor) from each detected object. Descriptor similarity is measured by cosine distance: a small distance indicates belonging to the same object.

5.2. Compared Models

Four models from three sources were integrated:

Model	Weight Source	Feature Size	Parameters	CPU FPS
OSNet x1_0	MSMT17 (HuggingFace)	512	2.2M	0.25
OSNet x0_25	MSMT17 (HuggingFace)	512	0.2M	4.32
OSNet-AIN x1_0	MSMT17 (HuggingFace)	512	2.2M	0.43
ResNet50- REID	ImageNet (torchvision)	2048	23.5M	1.75

OSNet is an architecture specifically designed for the REID task. The x0_25 version is a 4× reduced model. OSNet-AIN incorporates an attention mechanism. ResNet50-REID is implemented on the basis of ResNet50 (a 50-layer convolutional network) with modifications: last stride=1 and BNNeck (Batch Normalization Neck — a normalization layer between the feature extractor and the classifier).

Weights were obtained from three sources: - **Ultralytics** — YOLO detectors - **Hugging-Face** (via the torchreid library) — pre-trained OSNet - **torchvision** — ResNet50 with ImageNet weights

5.3. Evaluation Methodology

To isolate REID quality from detector quality, evaluation was conducted on GT (Ground Truth) bounding boxes. When using GT bboxes, DetJ approaches 100%, and differences in HOTA are determined exclusively by association quality. Parameters: max_cosine_distance=0.2, nn_budget=100.

5.4. Results on TUD-Campus (all 4 models)

Model	HOTA	DetJ	AssA	MOTA	IDF1	FPS
resnet50_reid	14.14	93.97	2.13	94.43	97.17	3.45
osnet_x1_0	14.14	93.97	2.13	94.43	97.17	0.25
osnet_x0_25	14.14	93.97	2.13	94.43	97.17	4.32

Model	HOTA	DetJ	AssA	MOTA	IDF1	FPS
osnet_ain_x1_0	14.14	93.97	2.13	94.43	97.17	0.43

The identical HOTA across all models is explained by the dominance of the Kalman filter when using GT bboxes: motion is predicted with high accuracy, and REID descriptors are used only in rare cases of trajectory intersection. On TUD-Campus (8 objects), such situations are virtually absent.

5.5. Full Results of ResNet50-REID on All 6 Videos

Video	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	14.14	93.97	2.13	94.43	97.17	3.45
TUD-Stadtmitte	8.82	96.59	0.80	97.06	95.64	1.96
KITTI-17	10.07	96.70	1.05	95.40	90.34	2.15
PETS09-S2L1	6.08	98.89	0.37	98.80	94.67	1.56
MOT16-09	6.71	98.61	0.46	98.30	96.20	0.52
MOT16-11	8.52	97.64	0.74	97.67	97.78	0.85
Average	9.06	97.07	0.93	96.94	95.30	1.75

HOTA was increased from 4.73 to 9.06 (+91.5%). The main contribution comes from ideal detection (GT bbox, DetJ=97%), but association was also improved (AssA 0.6 → 0.93) due to real REID descriptors.

5.6. Model Selection

Criterion	OSNet x0_25	ResNet50-REID	OSNet x1_0	OSNet-AIN x1_0
HOTA	14.14	14.14	14.14	14.14
FPS (CPU)	4.32	3.45	0.25	0.43
Parameters	0.2M	23.5M	2.2M	2.2M
Feature size	512	2048	512	512

The **OSNet x0_25** model was selected: identical accuracy to all models with the highest CPU FPS (4.32, 17× faster than OSNet x1_0) and the fewest parameters (0.2M). Pre-trained on MSMT17 — the largest REID dataset. Feature size of 512 (vs. 2048 for ResNet50) reduces computational costs for cosine distance.

6. Parameter Tuning

6.1. Full Pipeline

Integration of YOLO11s (detection) and OSNet x0_25 (REID) into a unified pipeline:

Image → YOLO11s (detection) → OSNet x0_25 (REID descriptors) → DeepSORT (tracking) → Re

Implemented in the script `evaluate_full_pipeline.py`, which performs a full video pass and computes HOTA, MOTA, IDF1, FPS.

6.2. Parameters

Parameter	Description	Default Value
<code>conf_threshold</code>	Detector confidence threshold; detections below the threshold are discarded	0.3
<code>nms_max_overlap</code>	NMS (Non-Maximum Suppression) threshold; eliminates duplicate bboxes	1.0
<code>max_cosine_distance</code>	Cosine distance threshold for track association	0.2
<code>nn_budget</code>	Number of recent descriptors stored per track	100
<code>max_age</code>	Maximum number of frames without detections before track deletion	30
<code>n_init</code>	Number of consecutive detections required to confirm a track	3
<code>max_iou_distance</code>	IoU (Intersection over Union) threshold for cascade matching	0.7

6.3. Experiments

Five experiments were conducted with varying parameters:

Experiment	<code>conf</code>	<code>cos_dist</code>	<code>n_init</code>	<code>max_age</code>	<code>budget</code>	HOTA	DetJ	AssA	MOTA	IDF1	FPS
default	0.3	0.2	3	30	100	6.48	57.36	0.82	59.42	67.63	2.94
tune1	0.2	0.3	2	50	100	6.01	52.47	0.78	47.67	60.96	2.55
tune2	0.4	0.15	3	30	50	6.63	58.10	0.84	60.58	65.93	2.24
tune3	0.35	0.25	1	20	100	6.56	57.75	0.83	59.84	66.50	2.11
tune4	0.4	0.2	3	30	100	6.61	58.63	0.83	61.18	67.94	2.16

6.4. Analysis

tune1 (conf=0.2, cos=0.3, n_init=2, max_age=50) — worst result. Lowering conf_threshold to 0.2 leads to an increase in false positives (FP): DetJ=52.47, MOTA=47.67. A low detection threshold degrades quality.

tune2 (conf=0.4, cos=0.15, budget=50) — maximum HOTA (6.63). Raising conf_threshold to 0.4 reduces the number of FPs. Lowering max_cosine_distance to 0.15 makes association stricter, however IDF1=65.93 — overly strict association splits tracks of a single object.

tune3 (conf=0.35, n_init=1, max_age=20) — n_init=1 (track confirmation after the first detection) does not improve HOTA (6.56) and reduces FPS to 2.11 due to an increased number of tracks.

tune4 (conf=0.4, remaining parameters at default) — optimal balance. HOTA=6.61, MOTA=61.18 (maximum), IDF1=67.94 (maximum).

6.5. Detailed Results of tune4

Video	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	9.60	50.18	1.84	49.86	65.84	3.07
TUD-Stadtmitte	7.80	73.12	0.83	76.64	83.31	2.48
KITTI-17	8.22	63.54	1.06	66.37	74.58	3.54
PETS09-S2L1	5.28	78.55	0.35	84.26	75.39	1.66
MOT16-09	3.98	39.70	0.40	40.14	47.95	1.09
MOT16-11	4.80	46.68	0.49	49.82	60.58	1.12
Average	6.61	58.63	0.83	61.18	67.94	2.16

6.6. Comparison with Baseline

Video	Baseline HOTA	tune4 HOTA	Improvement
TUD-Campus	7.95	9.60	+20.8%
TUD-Stadtmitte	5.33	7.80	+46.3%
KITTI-17	5.93	8.22	+38.6%
PETS09-S2L1	4.08	5.28	+29.4%
MOT16-09	2.59	3.98	+53.7%
MOT16-11	2.52	4.80	+90.5%
Average	4.73	6.61	+39.7%

HOTA exceeds the baseline on every sequence. The greatest improvement (90.5%) was achieved on MOT16-11 — the video with the highest object density, where high-quality REID descriptors are most effective.

The low results on MOT16-09 and MOT16-11 (HOTA 3.98 and 4.80) are due to the detector: at `img_size=640`, YOLO11s fails to detect distant objects. Increasing `img_size` to 1280 would potentially improve DetJ but slow inference by a factor of 2-3.

7. Autonomous Body REID

7.1. Problem Statement

Body REID is an additional task to tracking. Standard tracking assigns each object a track ID, however, when an object disappears from the frame and subsequently reappears, it is assigned a new track ID. Body REID addresses this problem: a database of identities is maintained, and for each track ID, the corresponding real identity is determined.

Example: an object with track ID=5 disappears from the frame for 100 frames, then reappears and receives track ID=12. Body REID establishes that ID=5 and ID=12 belong to the same identity (`identity=1`).

7.2. IdentityDatabase

IdentityDatabase is an identity database with k-nearest neighbors (kNN) search. Implemented using `sklearn.neighbors.NearestNeighbors` with cosine distance.

Algorithm: 1. A REID descriptor is extracted for each new track 2. A kNN search is performed for the nearest identity in the database 3. If the distance to the nearest neighbor is below `distance_threshold`, the track is assigned to an existing identity 4. If the distance exceeds the threshold, a new identity is created 5. The descriptor is added to the database for the corresponding identity

Centroid-based search (average descriptor per identity) is used (`use_centroid=True`), which provides greater accuracy with small data volumes. FIFO (First In First Out) eviction is implemented: when exceeding 50 descriptors per identity, the oldest are removed.

7.3. IdentityManager

IdentityManager is a layer above IdentityDatabase that manages track identification:

1. **Identification** — query to the database, obtaining (`identity_id`, `distance`)
2. **History recording** — for each track, a `TrackIdentityHistory` is stored with records (`frame_id`, `identity_id`, `distance`)
3. **Temporal window resolution** — over the last `T` frames (`temporal_window=30`), the final identity is determined by majority voting. This increases robustness to individual identification errors.
4. **Conflict resolution** — if two active tracks belong to the same identity, a reset strategy is applied: both tracks are reset and receive new identities upon the next identification.

7.4. Clustering Metrics

The body REID task is essentially a clustering task — grouping tracks by identities. The following metrics are used:

- **FMI (Fowlkes-Mallows Index)** — a measure of similarity between two partitions (obtained and ground-truth). Range [0, 1]; 1 — complete match.
- **Silhouette Coefficient** — a measure of cluster compactness and separability. Range [-1, 1]; 1 — optimal clustering.
- **Calinski-Harabasz Index** — ratio of inter-cluster variance to intra-cluster variance. Higher is better.
- **Purity** — the proportion of tracks correctly assigned to the dominant identity in each cluster.
- **Inverse Purity** — the proportion of clusters not containing tracks from different ground-truth identities.

7.5. Offline Clustering Results

Offline clustering: extraction of all descriptors from GT bboxes and threshold-based clustering (cosine distance ≤ 0.3):

Video	FMI	Silhouette	CH	Purity	GT IDs	Clusters
TUD-Campus	0.7899	0.4795	65.41	0.9415	8	11
TUD-Stadtmitte	0.7039	0.4278	84.59	0.8936	10	28
KITTI-17	0.6145	0.2430	15.73	0.9040	9	58
PETS09-S2L1	0.5089	0.2169	105.51	0.7568	19	79
MOT16-09	0.5584	0.3031	72.88	0.8188	43	262
MOT16-11	0.7407	0.3898	98.56	0.9066	80	230
Average	0.6527	0.3433	—	0.8702	—	—

The average FMI=0.65, with excessive fragmentation (over-segmentation) observed: on KITTI-17, 58 clusters were obtained for 9 ground-truth identities.

7.6. IdentityManager Results (Online)

Video	FMI	Purity	InvPurity	GT IDs	Assigned	Identities
TUD-Campus	0.8698	0.8747	0.8607	8	9	9
TUD-Stadtmitte	0.9482	0.9740	0.9057	10	14	17
KITTI-17	0.8984	0.8873	0.8425	9	14	31
PETS09-S2L1	0.7005	0.6727	0.8865	19	17	29
MOT16-09	0.8498	0.6789	0.7190	43	65	109
MOT16-11	0.9200	0.7829	0.8068	80	84	112
Average	0.8645	0.8118	0.8369	—	—	—

FMI was increased from 0.65 to 0.86 (+32.5%). The temporal window and conflict

resolution provide history accumulation and voting, which increases robustness to individual identification errors.

7.7. Threshold Comparison

Threshold	Avg FMI	Avg Purity	Avg InvPurity	Avg number of identities
0.2	0.7305	0.8939	0.6980	153 (fragmentation)
0.3	0.8645	0.8118	0.8369	51 (balance)

A threshold of 0.2 leads to excessive fragmentation: 153 identities instead of ~30 ground-truth. A threshold of 0.3 provides an optimal balance: FMI is 18% higher, fragmentation is minimal.

7.8. Segmentation Integration

For body REID, the yolol1s-seg detector was integrated — a segmentation variant of YOLO11s (in addition to bboxes, it returns a mask — pixel-level object segmentation). Using masks for cropping objects instead of bboxes excludes background from the descriptor, potentially improving REID quality.

Switching: `--detector yolol1s-seg`.

8. Overlay Videos

8.1. Purpose

Overlays — videos with superimposed bboxes and track IDs — are intended for visual assessment of algorithm performance. They allow identification of errors not visible in numerical metrics: track fragmentation, false detections, ID switches.

8.2. Generation Methodology

The script `generate_overlays.py` performs: 1. Reading frames from the original video 2. Reading tracking results from MOT files 3. Drawing colored bboxes with track IDs for each object 4. Assembling frames back into a video

Colors are assigned deterministically: each track ID corresponds to a fixed color (HSV with a step of 0.41), ensuring visual consistency.

Three types of videos were generated: - **Comparison** — side-by-side: baseline on the left, best implementation on the right - **Baseline** — standalone video with the baseline - **Best** — standalone video with the best implementation

8.3. Result

18 videos were generated (6 comparison + 6 baseline + 6 best) in the overlays/ directory: - overlays/comparison/ — side-by-side comparisons - overlays/baseline/ — baseline only - overlays/best/ — best implementation only

During generation, resolution differences had to be accounted for: KITTI-17 (1224×370) differs from the others (640×480 or 1920×1080). Frame dimensions are determined by reading the first image rather than from seqinfo.ini, since metadata does not always match the actual values.

9. Performance

9.1. CPU Speed

The full pipeline runs at 2.16 FPS on CPU, which is below the ≥ 5 FPS requirement. The reason is the high computational cost of neural network inference on CPU.

9.2. Time Distribution by Component

Component	CPU, time/frame	Share
YOLO11s (detection)	~0.11s	~24%
OSNet x0_25 (REID)	~0.23s	~50%
DeepSORT (tracking)	~0.01s	~2%
Other (I/O, preprocessing)	~0.11s	~24%
Total	~0.46s	100%

The primary bottleneck is REID inference (50% of time). OSNet x0_25 processes each crop (extracted bbox) separately; with 20 objects per frame, 20 model invocations are performed.

9.3. GPU Performance Estimate

On Google Colab with a GPU (T4 or V100), the following is expected: - YOLO11s: ~100+ FPS (vs. 8.77 on CPU) — ~10× speedup - OSNet x0_25: ~200+ FPS (vs. 4.32 on CPU) — ~50× speedup - Full pipeline: ≥ 5 FPS with margin (expected 40+ FPS)

On GPU, REID batching (processing all crops in a single model invocation) is possible, which would provide additional speedup.

10. Conclusion

Work Performed

The DeepSORT algorithm (2017) was modernized by replacing outdated components with contemporary neural network architectures: - POI detector → **YOLO11s** (mAP 47.0, 8.77 FPS on CPU) - Random REID vectors → **OSNet x0_25** (512-dim, 0.2M parameters, 4.32 FPS on CPU) - Integrated **YOLO11s-seg** model for segmentation - Implemented autonomous body REID system (IdentityDatabase + IdentityManager) - Tuned parameters: `conf_threshold=0.4`, remaining at default values - Generated 18 overlay videos for visual comparison

Key Finding

Model selection contributes more to quality than parameter tuning. The transition from random REID vectors to the trained OSNet x0_25 model yielded +37% HOTA. Changing `conf_threshold` from 0.3 to 0.4 yielded only +2%. Under limited resources, it is advisable to invest in model selection and integration rather than hyperparameter search.

Difficulties

The main difficulty arose in interpreting the identical HOTA across all REID models when evaluated on GT bboxes. A code bug was initially suspected; analysis revealed that with ideal bboxes, the Kalman filter predicts motion with sufficient accuracy, and REID descriptors are used only during trajectory intersections — situations that are rare on sparsely populated sequences.

Integration of OSNet via `torchreid` required manual weight loading from HuggingFace, as the `torchreid` library is not installable via pip. ResNet50-REID was implemented from scratch based on `torchvision`: last stride changed to 1, BNNeck added.

Potential Improvements

1. **GPU utilization** — on CPU, REID inference accounts for 50% of time; a single pass over MOT16-11 (900 frames) takes ~7 minutes.
2. **Increasing `img_size` to 1280** for MOT16-09 and MOT16-11 — would improve detection of distant objects.
3. **REID batching** — processing all crops in a frame in a single model invocation.
4. **Augmentations during REID evaluation** — horizontal flip, color jitter for robustness testing.

Final Results

Metric	Baseline	Best	Improvement
HOTA	4.73	6.61	+39.7%
MOTA	36.48	61.18	+67.7%
IDF1	35.96	67.94	+88.9%

Metric	Baseline	Best	Improvement
Body REID FMI	—	0.86	—
FPS (CPU)	334.57*	2.16	—
FPS (GPU, estimate)	—	40+	—

* Baseline: pre-loaded detections + random vectors (no neural network inference).

HOTA exceeds the baseline on each of the 6 sequences — from +20.8% (TUD-Campus) to +90.5% (MOT16-11), confirming the robustness of the modernization across various conditions.